Exercice 2 — Gestion des étudiants et des résultats académiques

Correction complète : analyse, UML et implémentation C++

ZINSOU DIEUDONNE
Matricule : 11038STI25

1 Identification des classes

On repère les classes en cherchant les *noms* importants de l'énoncé, ceux qui ont des données propres et un comportement.

Classe	Attributs	Responsabilités principales
`Etudiant`	matricule, nom, prénom, niveau (L1/L2/L3), filière, notes	ajouter une note, moyenne d'une UE, validation d'une UE, moyenne générale, relevé
`Filiere`	code, nom, liste des UE	ajouter une UE, total des crédits
`UE`	code, intitulé, crédit, liste des matières	ajouter une matière, somme des coefficients
`Matiere`	code, intitulé, coefficient	fournir ses informations
`Note`	valeur ($0 \leq v \leq 20$), matière concernée	contrôler et modifier la valeur

Le niveau (L1, L2, L3) n'est pas une classe : c'est un ensemble fini de valeurs, on le représente par une énumération `enum class Niveau`.

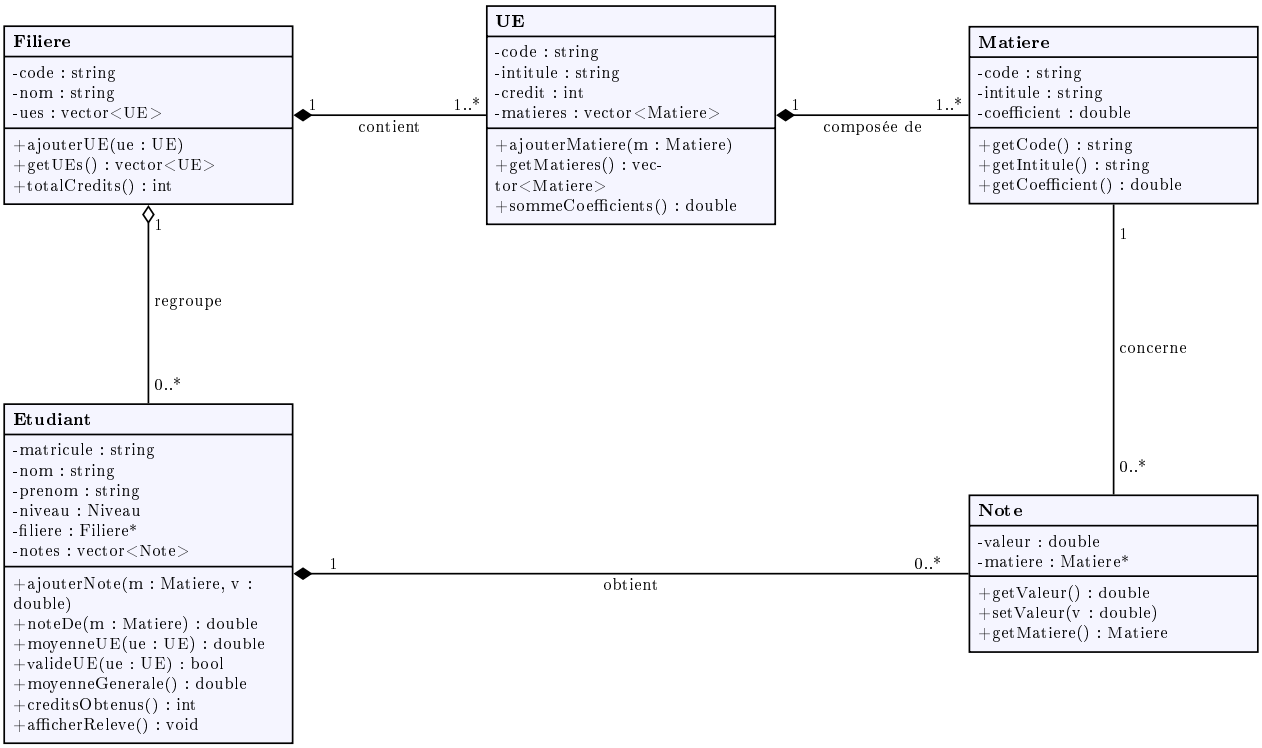
2 Relations entre les classes

Relation	Type	Multiplicité	Justification
Filiere → UE	composition	1 1..∗	une UE n'existe pas hors de sa filière
UE → Matiere	composition	1 1..∗	une matière appartient à une seule UE, qui en a au moins une
Filiere → Etudiant	agrégation	1 0..∗	l'étudiant appartient à une filière mais existe indépendamment d'elle
Etudiant → Note	composition	1 0..∗	les notes d'un étudiant disparaissent avec lui
Matiere ↔ Note	association	1 0..∗	une note porte sur une matière ; la matière existe sans les notes

Rappel.

- **Composition** (losange plein) : lien fort, le « tout » possède ses « parties » et leur durée de vie en dépend. En C++ : l'objet est stocké *par valeur* (`vector<Matiere>`).
- **Agrégation** (losange vide) : lien faible, la partie survit sans le tout. En C++ : on garde un *pointeur ou une référence* (`const Filiere*`).
- **Association** : simple lien entre deux objets (ici `Note` garde un pointeur vers `Matiere`).

3 Diagramme de classes UML



Lecture : une filière contient au moins une UE ; une UE est composée d'au moins une matière ; une filière regroupe zéro ou plusieurs étudiants ; un étudiant obtient zéro ou plusieurs notes ; chaque note concerne exactement une matière.

4 Règles de calcul retenues

- **Moyenne d'une UE** (moyenne pondérée par les coefficients des matières) :

$$\text{moy}(UE) = \frac{\sum_{m \in UE} \text{note}_m \times \text{coef}_m}{\sum_{m \in UE} \text{coef}_m}$$

- **Validation d'une UE** : l'UE est validée si $\text{moy}(UE) \geq 10$.
- **Moyenne générale** (moyenne des UE pondérée par les crédits) :

$$\text{MG} = \frac{\sum_{UE} \text{moy}(UE) \times \text{crédit}(UE)}{\sum_{UE} \text{crédit}(UE)}$$

- **Note manquante** : elle compte pour 0 dans les calculs.
- **Mention** : ≥ 16 Très bien, ≥ 14 Bien, ≥ 12 Assez bien, ≥ 10 Passable, sinon Ajourné.

5 Implémentation en C++

Le projet est découpé en six fichiers : `Matiere.h`, `UE.h`, `Filiere.h`, `Note.h`, `Etudiant.h` et `main.cpp`. L'**encapsulation** est assurée par des attributs `private`, des accesseurs en lecture et des contrôles dans les constructeurs et modificateurs (note entre 0 et 20, coefficient et crédit positifs).

5.1 Classe Matiere

La plus petite brique : un code, un intitulé et un coefficient.

Listing 1: `Matiere.h`

```cpp
#ifndef MATIERE_H
#define MATIERE_H

#include <string>
#include <stdexcept>

// Une matiere : plus petite unite d'enseignement evaluee (note + coefficient)
class Matiere {
private:
    std::string code;
    std::string intitule;
    double coefficient;

public:
    Matiere(const std::string& code, const std::string& intitule, double coefficient)
        : code(code), intitule(intitule), coefficient(coefficient) {
        if (coefficient <= 0)
            throw std::invalid_argument("Le coefficient doit etre strictement positif");
    }

    const std::string& getCode() const     { return code; }
    const std::string& getIntitule() const { return intitule; }
    double getCoefficient() const          { return coefficient; }
};

#endif
```

5.2 Classe UE

Composition : les matières sont stockées par valeur dans un `vector<Matiere>`.

Listing 2: `UE.h`

```cpp
#ifndef UE_H
#define UE_H

#include <string>
#include <vector>
#include <stdexcept>
#include "Matiere.h"

// Une unite d'enseignement : possede un credit et se compose de matieres
// COMPOSITION : les matieres sont stockees PAR VALEUR dans l'UE
class UE {
private:
    std::string code;
    std::string intitule;
    int credit;
    std::vector<Matiere> matieres;

public:
    UE(const std::string& code, const std::string& intitule, int credit)
```

```
        : code(code), intitule(intitule), credit(credit) {
        if (credit <= 0)
            throw std::invalid_argument("Le credit doit etre strictement positif");
    }

    void ajouterMatiere(const Matiere& m) { matieres.push_back(m); }

    const std::string& getCode() const            { return code; }
    const std::string& getIntitule() const        { return intitule; }
    int getCredit() const                         { return credit; }
    const std::vector<Matiere>& getMatieres() const { return matieres; }

    double sommeCoefficients() const {
        double s = 0;
        for (const Matiere& m : matieres)
            s += m.getCoefficient();
        return s;
    }
};

#endif
```

5.3 Classe Filiere

Composition : les UE sont stockées par valeur dans un `vector<UE>`.

Listing 3: `Filiere.h`

```
#ifndef FILIERE_H
#define FILIERE_H

#include <string>
#include <vector>
#include "UE.h"

// Une filiere : regroupe plusieurs UE
// COMPOSITION : les UE sont stockees PAR VALEUR dans la filiere
class Filiere {
private:
    std::string code;
    std::string nom;
    std::vector<UE> ues;

public:
    Filiere(const std::string& code, const std::string& nom)
        : code(code), nom(nom) {}

    void ajouterUE(const UE& ue) { ues.push_back(ue); }

    const std::string& getCode() const     { return code; }
    const std::string& getNom() const      { return nom; }
    const std::vector<UE>& getUEs() const  { return ues; }

    int totalCredits() const {
        int total = 0;
        for (const UE& ue : ues)
            total += ue.getCredit();
        return total;
    }
};

#endif
```

5.4 Classe Note

Association avec `Matiere` (pointeur). La valeur est contrôlée entre 0 et 20.

Listing 4: `Note.h`

```cpp
#ifndef NOTE_H
#define NOTE_H

#include <stdexcept>
#include "Matiere.h"

// Une note : valeur (0 a 20) obtenue dans UNE matiere
// ASSOCIATION avec Matiere : on garde un pointeur (la matiere existe independamment)
class Note {
private:
    const Matiere* matiere;
    double valeur;

    static void verifier(double v) {
        if (v < 0.0 || v > 20.0)
            throw std::invalid_argument("Une note doit etre comprise entre 0 et 20");
    }

public:
    Note(const Matiere& m, double valeur) : matiere(&m), valeur(valeur) {
        verifier(valeur);
    }

    const Matiere& getMatiere() const { return *matiere; }
    double getValeur() const          { return valeur; }

    void setValeur(double v) {
        verifier(v);
        valeur = v;
    }
};

#endif
```

5.5 Classe Etudiant (avec les calculs et le relevé)

Agrégation avec `Filiere` (pointeur), composition avec `Note` (`vector<Note>`). C'est ici que se trouvent `moyenneUE`, `valideUE`, `moyenneGenerale` et `afficherReleve`.

Listing 5: `Etudiant.h`

```cpp
#ifndef ETUDIANT_H
#define ETUDIANT_H

#include <string>
#include <vector>
#include <iostream>
#include <iomanip>
#include "Filiere.h"
#include "Note.h"

enum class Niveau { L1, L2, L3 };

inline std::string niveauToString(Niveau n) {
    switch (n) {
        case Niveau::L1: return "L1";
        case Niveau::L2: return "L2";
        case Niveau::L3: return "L3";
    }
```

```
    return "?";
}

// Un etudiant appartient a une filiere (AGREGATION : pointeur vers la filiere)
// et possede ses notes (COMPOSITION : vector<Note> stocke par valeur)
class Etudiant {
private:
    std::string matricule;
    std::string nom;
    std::string prenom;
    Niveau niveau;
    const Filiere* filiere;
    std::vector<Note> notes;

    static std::string mention(double moyenne) {
        if (moyenne >= 16) return "Tres bien";
        if (moyenne >= 14) return "Bien";
        if (moyenne >= 12) return "Assez bien";
        if (moyenne >= 10) return "Passable";
        return "Ajourne";
    }

public:
    Etudiant(const std::string& matricule, const std::string& nom,
             const std::string& prenom, Niveau niveau, const Filiere& filiere)
        : matricule(matricule), nom(nom), prenom(prenom),
          niveau(niveau), filiere(&filiere) {}

    // ---------- Getters ----------
    const std::string& getMatricule() const { return matricule; }
    const std::string& getNom() const       { return nom; }
    const std::string& getPrenom() const    { return prenom; }
    Niveau getNiveau() const                { return niveau; }
    const Filiere& getFiliere() const       { return *filiere; }
    const std::vector<Note>& getNotes() const { return notes; }

    // ---------- Gestion des notes ----------
    // Ajoute la note, ou la remplace si l'etudiant en a deja une dans cette matiere
    void ajouterNote(const Matiere& m, double valeur) {
        for (Note& n : notes) {
            if (n.getMatiere().getCode() == m.getCode()) {
                n.setValeur(valeur);
                return;
            }
        }
        notes.push_back(Note(m, valeur));
    }

    // Retourne la note dans la matiere, ou -1 si l'etudiant n'en a pas
    double noteDe(const Matiere& m) const {
        for (const Note& n : notes)
            if (n.getMatiere().getCode() == m.getCode())
                return n.getValeur();
        return -1.0;
    }

    // ---------- Calculs ----------
    // Moyenne ponderee (par les coefficients) d'une UE.
    // Une note manquante compte pour 0.
    double moyenneUE(const UE& ue) const {
        double somme = 0.0, coefs = 0.0;
        for (const Matiere& m : ue.getMatieres()) {
            double n = noteDe(m);
            if (n < 0) n = 0.0;
            somme += n * m.getCoefficient();
```

```
            coefs += m.getCoefficient();
        }
        return (coefs > 0) ? somme / coefs : 0.0;
    }

    // Une UE est validee si sa moyenne est >= seuil (10 par defaut)
    bool valideUE(const UE& ue, double seuil = 10.0) const {
        return moyenneUE(ue) >= seuil;
    }

    // Moyenne generale = moyenne des moyennes d'UE ponderee par les credits
    double moyenneGenerale() const {
        double somme = 0.0;
        int credits = filiere->totalCredits();
        for (const UE& ue : filiere->getUEs())
            somme += moyenneUE(ue) * ue.getCredit();
        return (credits > 0) ? somme / credits : 0.0;
    }

    int creditsObtenus() const {
        int total = 0;
        for (const UE& ue : filiere->getUEs())
            if (valideUE(ue))
                total += ue.getCredit();
        return total;
    }

    // ---------- Affichage ----------
    void afficherReleve() const {
        const std::string trait(62, '=');
        std::cout << std::fixed << std::setprecision(2);
        std::cout << trait << "\n";
        std::cout << "                        RELEVE DE NOTES\n";
        std::cout << trait << "\n";
        std::cout << "Etudiant  : " << nom << " " << prenom
                  << " (" << matricule << ")\n";
        std::cout << "Filiere   : " << filiere->getNom()
                  << "   Niveau : " << niveauToString(niveau) << "\n";
        std::cout << trait << "\n\n";

        for (const UE& ue : filiere->getUEs()) {
            std::cout << "UE " << ue.getCode() << " - " << ue.getIntitule()
                      << " (" << ue.getCredit() << " credits)\n";
            for (const Matiere& m : ue.getMatieres()) {
                double n = noteDe(m);
                std::cout << "   " << std::left << std::setw(30) << m.getIntitule()
                          << " coef " << std::right << std::setw(5) << m.getCoefficient()
                          << "   note : ";
                if (n < 0) std::cout << std::setw(6) << "--";
                else       std::cout << std::setw(6) << n;
                std::cout << "\n";
            }
            std::cout << "   >> Moyenne UE : " << moyenneUE(ue) << " / 20  ->  "
                      << (valideUE(ue) ? "VALIDEE" : "NON VALIDEE") << "\n\n";
        }

        double moy = moyenneGenerale();
        std::cout << trait << "\n";
        std::cout << "Moyenne generale : " << moy << " / 20\n";
        std::cout << "Credits obtenus  : " << creditsObtenus()
                  << " / " << filiere->totalCredits() << "\n";
        std::cout << "Mention          : " << mention(moy) << "\n";
        std::cout << trait << "\n";
    }
};
```

```
149
150 #endif
```

5.6 Programme principal

On construit d'abord la filière complète, puis on crée les étudiants et leurs notes.

Listing 6: `main.cpp`

```cpp
#include <iostream>
#include "Etudiant.h"

int main() {
    // ----- 1. Construction de la filiere (AVANT de creer les notes) -----
    Filiere mi("MI", "Mathematiques-Informatique");

    UE algo("UE11", "Algorithmique et programmation", 6);
    algo.ajouterMatiere(Matiere("INF111", "Structures de donnees", 2));
    algo.ajouterMatiere(Matiere("INF112", "Programmation C", 1));

    UE analyse("UE12", "Analyse", 4);
    analyse.ajouterMatiere(Matiere("MAT121", "Analyse vectorielle", 2));
    analyse.ajouterMatiere(Matiere("MAT122", "Analyse complexe", 2));

    UE proba("UE13", "Probabilites et statistiques", 5);
    proba.ajouterMatiere(Matiere("MAT131", "Probabilites", 2));
    proba.ajouterMatiere(Matiere("MAT132", "Statistiques", 1.5));

    mi.ajouterUE(algo);
    mi.ajouterUE(analyse);
    mi.ajouterUE(proba);

    // Reperage des matieres (references vers les objets de la filiere,
    // valables tant que la filiere n'est plus modifiee)
    const auto& ues = mi.getUEs();
    const Matiere& sd   = ues[0].getMatieres()[0];
    const Matiere& c    = ues[0].getMatieres()[1];
    const Matiere& av   = ues[1].getMatieres()[0];
    const Matiere& ac   = ues[1].getMatieres()[1];
    const Matiere& prob = ues[2].getMatieres()[0];
    const Matiere& stat = ues[2].getMatieres()[1];

    // ----- 2. Etudiants et notes -----
    Etudiant e1("21MI001", "AKOTO", "Koffi", Niveau::L2, mi);
    e1.ajouterNote(sd, 14);
    e1.ajouterNote(c, 16);
    e1.ajouterNote(av, 11.5);
    e1.ajouterNote(ac, 9);
    e1.ajouterNote(prob, 12);
    e1.ajouterNote(stat, 13);

    Etudiant e2("21MI002", "BIO", "Aicha", Niveau::L2, mi);
    e2.ajouterNote(sd, 8);
    e2.ajouterNote(c, 10);
    e2.ajouterNote(av, 7);
    e2.ajouterNote(ac, 12);
    e2.ajouterNote(prob, 9.5);
    // pas de note en statistiques -> comptee 0

    // ----- 3. Resultats -----
    e1.afficherReleve();
    std::cout << "\n\n";
    e2.afficherReleve();

```

```
    return 0;
}
```

Attention (pointeurs). `Note` et `Etudiant` gardent des pointeurs vers des objets de la filière. Il faut donc *construire entièrement la filière avant* de créer les notes (sinon un `push_back` pourrait réallouer le `vector` et invalider les pointeurs).

6 Compilation et exécution

```
g++ -std=c++17 -Wall -Wextra -o exo2 main.cpp
./exo2
```

Résultat obtenu (deux étudiants : le premier valide tout, le second échoue) :

```
============================================================
                    RELEVE DE NOTES
============================================================
Etudiant  : AKOTO Koffi (21MI001)
Filiere   : Mathematiques-Informatique   Niveau : L2
============================================================

UE UE11 - Algorithmique et programmation (6 credits)
   Structures de donnees          coef  2.00   note :  14.00
   Programmation C                coef  1.00   note :  16.00
   >> Moyenne UE : 14.67 / 20  ->  VALIDEE

UE UE12 - Analyse (4 credits)
   Analyse vectorielle            coef  2.00   note :  11.50
   Analyse complexe               coef  2.00   note :   9.00
   >> Moyenne UE : 10.25 / 20  ->  VALIDEE

UE UE13 - Probabilites et statistiques (5 credits)
   Probabilites                   coef  2.00   note :  12.00
   Statistiques                   coef  1.50   note :  13.00
   >> Moyenne UE : 12.43 / 20  ->  VALIDEE

============================================================
Moyenne generale : 12.74 / 20
Credits obtenus  : 15 / 15
Mention          : Assez bien
============================================================



============================================================
                    RELEVE DE NOTES
============================================================
Etudiant  : BIO Aicha (21MI002)
Filiere   : Mathematiques-Informatique   Niveau : L2
============================================================

UE UE11 - Algorithmique et programmation (6 credits)
   Structures de donnees          coef  2.00   note :   8.00
   Programmation C                coef  1.00   note :  10.00
   >> Moyenne UE : 8.67 / 20  ->  NON VALIDEE

UE UE12 - Analyse (4 credits)
   Analyse vectorielle            coef  2.00   note :   7.00
   Analyse complexe               coef  2.00   note :  12.00
   >> Moyenne UE : 9.50 / 20  ->  NON VALIDEE

UE UE13 - Probabilites et statistiques (5 credits)
   Probabilites                   coef  2.00   note :   9.50
   Statistiques                   coef  1.50   note :     --
   >> Moyenne UE : 5.43 / 20  ->  NON VALIDEE

============================================================
Moyenne generale : 7.81 / 20
Credits obtenus  : 0 / 15
Mention          : Ajourne
============================================================
```

Vérification à la main (étudiant 1, UE11). $\dfrac{14 \times 2 + 16 \times 1}{2+1} = \dfrac{44}{3} \approx 14{,}67$, donc UE validée.

Moyenne générale : $\dfrac{14{,}67 \times 6 + 10{,}25 \times 4 + 12{,}43 \times 5}{15} \approx 12{,}74$.